

NOVAC Internship – Task 8 Daily Documentation

Task: Advanced RAG Pipeline Optimization, Mistral AI Integration, Semantic Chunking Improvements, Persistent Knowledge Base Management, and Conversational Retrieval Accuracy Enhancement

Overview

Today's work focused on transforming the existing conversational semantic retrieval system into a more advanced Retrieval-Augmented Generation (RAG) architecture using Mistral AI for contextual response generation.

The primary objective was to improve:

- response quality
- semantic retrieval accuracy
- contextual understanding
- conversational naturalness
- chunk management workflows

By the end of today's implementation, the system successfully supported:

- Mistral AI-powered response generation
- advanced semantic chunking
- overlapping chunk context handling
- embedding regeneration during chunk updates
- persistent knowledge-base style storage
- conversational RAG workflows
- cleaner chatbot UI rendering
- improved retrieval precision

Resources and References Used

- Mistral AI Documentation
- Sentence Transformers Documentation
- FastAPI Documentation
- MongoDB Documentation
- Vector Embedding Retrieval Concepts
- RAG Architecture Concepts
- Whisper Documentation
- React Documentation

Concepts Learned

Retrieval-Augmented Generation (RAG)

Learned how modern AI assistants combine:

- semantic retrieval
- vector embeddings
- LLM generation

to produce:

- grounded
- contextual
- human-readable responses

Implemented:

- contextual response generation

- retrieval-to-generation pipeline
- prompt-based context grounding
- conversational AI generation workflows

Mistral AI Integration

Learned how external LLM APIs integrate into backend AI systems.

Implemented:

- Mistral API authentication
- environment variable handling
- API-based response generation
- contextual prompt construction
- conversational answer formatting

Understood:

- LLM prompting workflows
- context injection
- prompt engineering fundamentals
- grounding responses using retrieved chunks

Advanced Semantic Chunking

Improved chunking architecture significantly.

Previous chunking:

- larger mixed-topic chunks
- inconsistent semantic boundaries
- weaker retrieval precision

New chunking architecture:

- ~500 character chunks
- overlapping contextual windows
- cleaner semantic segmentation
- improved retrieval targeting

Implemented:

- chunk overlap handling
- context continuity
- semantic chunk refinement
- smaller focused retrieval units

Embedding Regeneration

Implemented:

- automatic embedding regeneration
- vector synchronization workflows
- retrieval memory reset after updates

This ensured:

- semantic consistency
- accurate vector similarity

- updated retrieval behavior

Persistent Knowledge Base Architecture

Implemented:

- non-destructive uploads
- duplicate chunk prevention
- cumulative knowledge storage
- editable knowledge management

Understood:

- long-term chatbot memory systems
- knowledge accumulation workflows
- persistent vector database concepts

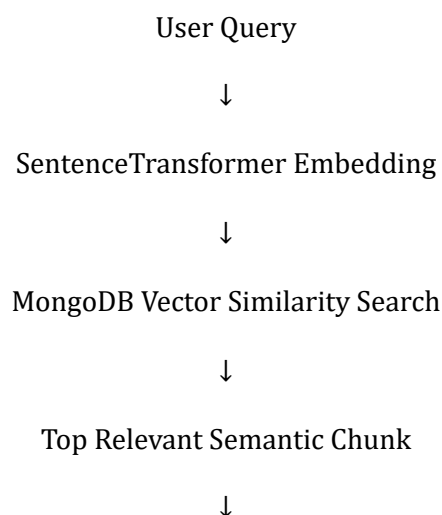
Libraries, Modules, and Components Learned

- **Mistral AI SDK:** Used for contextual response generation.
- **Sentence Transformers:** Used for semantic embedding generation.
- **FastAPI:** Used for backend API development.
- **MongoDB:** Used for vector storage and chunk persistence.
- **Whisper:** Used for voice-to-text conversion.
- **React.js:** Used for conversational chatbot frontend.
- **dotenv:** Used for API key management.
- **cosine_similarity:** Used for semantic vector comparison.

Practical Implementation

- Mistral AI API integration
- contextual response generation pipeline
- semantic-to-LLM retrieval workflow
- improved chunk segmentation logic
- overlapping chunk retrieval strategy
- embedding regeneration during chunk editing
- persistent knowledge-base storage
- duplicate chunk prevention
- conversational retrieval memory reset
- improved retrieval quality architecture
- cleaner chatbot response rendering
- removal of similarity score UI

Architecture Understood



Mistral AI Context Injection



Human-Like Conversational Response



Frontend Chatbot Rendering

Challenges Faced

Mistral SDK Version Conflicts

Encountered:

- import errors
- SDK version mismatches
- incompatible client initialization

Resolved by:

- correcting SDK imports
- updating API client usage
- resolving package compatibility issues

Python Cache Issues

Faced:

- stale imports
- cached module loading
- incorrect reload behavior

Resolved using:

- cache cleanup
- proper server restarts
- module refresh workflows

Retrieval Accuracy Issues

Initial responses were:

- vague
- overly broad
- contextually weak

Resolved through:

- smaller semantic chunks
- overlap-based chunking
- cleaner semantic segmentation

Embedding Synchronization Problems

Discovered:

- edited chunks retained old embeddings

- semantic retrieval became inconsistent

Resolved by:

- automatic embedding regeneration
- synchronized vector updates
- conversational memory resets

Conclusion

Today's work significantly upgraded the chatbot into a much more advanced Retrieval-Augmented Generation (RAG) system capable of producing contextual AI-generated responses using semantic retrieval and Mistral AI generation.

The platform now supports:

- contextual AI-generated answers
- advanced semantic retrieval
- optimized chunking workflows
- persistent knowledge management
- editable vector-based retrieval
- conversational RAG architecture
- voice-enabled querying
- semantic memory handling

The concepts learned today form the foundation of:

- production-grade RAG systems
- enterprise AI assistants
- contextual retrieval architectures
- vector database systems
- conversational LLM applications
- AI-powered document intelligence systems

Prepared By

Divesh Kumar S